

# Agentes RAG — Análisis completo de la Sesión 13

**Fuente base:** *Agentes IA — RAG* — Módulo 5 (Herramientas para Orquestación), Programa en Diseño e Implementación de Agentes IA, UTEC Posgrado. Dictada por MEng. Boris Alzamora (mismo docente de la Sesión 9).

**Nota técnica:** el PDF original ( `SES13_M5_Langchain_Agents_RAG.pdf` , exportado desde PowerPoint) no tenía capa de texto en la mayoría de sus diapositivas — son diagramas e imágenes. Este documento se generó extrayendo el texto disponible y **renderizando e interpretando visualmente** las diapositivas puramente gráficas (embeddings, arquitecturas RAG/Agentic RAG/GraphRAG, RAGAS, chunking).

**Complementado con:** investigación propia sobre el paper original de RAG (Lewis et al., 2020), la técnica de *Contextual Retrieval* de Anthropic (2024), GraphRAG de Microsoft Research, y el framework de evaluación RAGAS.

## 1. Objetivos y Agenda

### Objetivos declarados:

1. Comprender el funcionamiento interno de **RAG** (*Retrieval-Augmented Generation*, generación aumentada por recuperación).
2. Implementar un *RAG Agent* usando **LangChain**.

### Agenda — Parte 1 (fundamentos):

|                                                          |
|----------------------------------------------------------|
| #   Tema                                                 |
| ---  ---                                                 |
| 1   Representación en <i>embeddings</i>                  |
| 2   <i>Vector Databases</i> (bases de datos vectoriales) |
| 3   RAG Architecture                                     |

### Agenda — Parte 2 (patrones avanzados):

|                                            |
|--------------------------------------------|
| #   Tema                                   |
| ---  ---                                   |
| 4   RAG (evaluación y práctica con Chroma) |
| 5   Agentic RAG                            |
| 6   GraphRAG                               |
| 7   Contextual Retrieval                   |

## 2. Vector Embeddings — las tres familias

Un **vector embedding** es una representación numérica de un dato (palabra, frase, imagen, documento) en un espacio multidimensional, donde objetos semánticamente similares quedan **cerca** entre sí (ej. "rey" y "reina" tienen embeddings cercanos). El material clasifica los métodos de generación de embeddings en tres familias, en orden creciente de sofisticación:

### 2.1 Frequency Based (basados en frecuencia)

Se apoyan en estadísticas simples del texto — no requieren entrenamiento complejo.

| Método                                                                                                                      | Qué hace                                                                                                                                                          |
|-----------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>BoW</b> ( <i>Bag of Words</i> , bolsa de palabras)                                                                       | Representa un texto como un vector donde cada dimensión es una palabra del vocabulario y el valor es su frecuencia. No captura orden ni contexto.                 |
| <b>TF-IDF</b> ( <i>Term Frequency-Inverse Document Frequency</i> , frecuencia de término – frecuencia inversa de documento) | Mejora BoW ponderando cada palabra según su frecuencia en el documento y su rareza en el corpus — palabras comunes ("el", "de") pesan menos que las poco comunes. |

**Ventajas:** simples, útiles para clasificación de texto o búsqueda de documentos. **Limitaciones:** no capturan relaciones semánticas ni contexto; vectores dispersos y de alta dimensión.

### 2.2 Prediction Based (basados en predicción)

Entrenan modelos para predecir palabras en función de su contexto, aprendiendo representaciones densas y semánticas.

| Método                                                         | Qué hace                                                                                                              |
|----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Word2Vec — CBOW</b><br>( <i>Continuous Bag of Words</i> )   | Predice una palabra a partir de su contexto.                                                                          |
| <b>Word2Vec — Skip-gram</b>                                    | Predice el contexto a partir de una palabra.                                                                          |
| <b>GloVe</b> ( <i>Global Vectors for Word Representation</i> ) | Combina estadísticas globales de co-ocurrencia con aprendizaje predictivo.                                            |
| <b>FastText</b>                                                | Extiende Word2Vec incorporando subpalabras — mejora la representación de palabras raras o morfológicamente complejas. |
| <b>Transformers</b> (BERT, GPT)                                | Generan embeddings contextualizados.                                                                                  |

**Ventajas:** capturan relaciones semánticas profundas; vectores densos y de baja dimensión; permiten analogías (ej. *rey – hombre + mujer ≈ reina*). **Limitaciones:** requieren entrenamiento y recursos computacionales; más difíciles de interpretar.

### 2.3 Contextual Based (basados en contexto)

A diferencia de los dos métodos anteriores, generan una representación **distinta para la misma palabra según el contexto** en el que aparece (ej. "banco" en "*el banco del parque*" vs. "*el banco financiero*" tiene embeddings diferentes).

| Modelo                                                                         | Característica                                                                            |
|--------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <b>BERT</b> ( <i>Bidirectional Encoder Representations from Transformers</i> ) | Embeddings bidireccionales — considera contexto a la izquierda y derecha de cada palabra. |
| <b>GPT</b> ( <i>Generative Pretrained Transformer</i> )                        | Embeddings unidireccionales (izquierda a derecha), útiles para tareas generativas.        |
| <b>RoBERTa, T5, DeBERTa</b>                                                    | Variantes/mejoras sobre BERT con distintos objetivos de entrenamiento.                    |

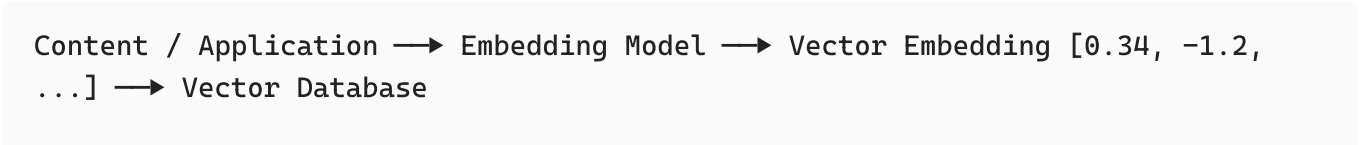
**Por qué esto es la base de los sistemas RAG modernos:** un sistema RAG necesita entender el significado completo de una consulta o documento para recuperar la información correcta — solo los embeddings contextuales logran eso de forma confiable. **Limitación:** requieren mucha capacidad computacional y son difíciles de interpretar.

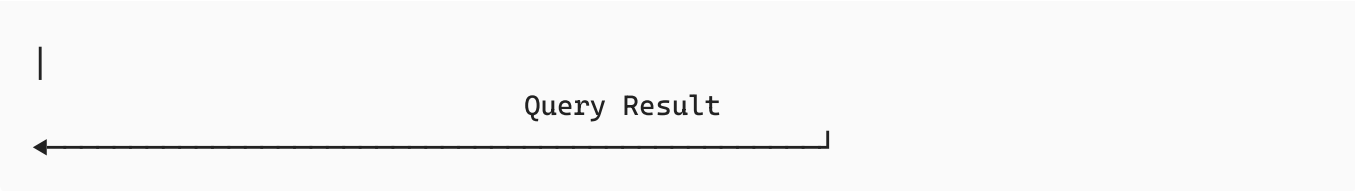
**Progresión del material:** frecuencia → predicción → contexto es, igual que en la Sesión 9 (clases legacy de memoria → estrategias modernas), una escalera de sofisticación: cada familia resuelve una limitación real de la anterior a costa de más cómputo.

## 3. Vector Database — cómo se almacena y busca

Una **base de datos vectorial** combina varios algoritmos para resolver la búsqueda por **vecino más cercano aproximado** (*Approximate Nearest Neighbour*, ANN) — no exacto, porque comparar contra absolutamente todos los vectores almacenados sería demasiado lento a gran escala. El *trade-off* central es **precisión vs. velocidad**: mientras más preciso el resultado, más lenta la consulta.

**El pipeline de tres pasos:**





| Paso                                      | Qué hace                                                                                                                                                                                                                                                                                                                                                  |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Indexing</b> (indexado)                | Asigna los vectores a una estructura de datos que permite búsqueda rápida, usando algoritmos como <b>PQ</b> ( <i>Product Quantization</i> , cuantización de producto), <b>LSH</b> ( <i>Locality Sensitive Hashing</i> , hashing sensible a la localidad) o <b>HNSW</b> ( <i>Hierarchical Navigable Small World</i> , mundo pequeño navegable jerárquico). |
| <b>Querying</b> (consulta)                | Compara el vector de consulta con los vectores indexados usando una métrica de similitud (textual, semántica o híbrida) para encontrar los vecinos más cercanos.                                                                                                                                                                                          |
| <b>Post Processing</b> (posprocesamiento) | Opcionalmente, reclasifica ( <i>re-rank</i> ) los vecinos recuperados usando una medida de similitud diferente antes de entregar el resultado final.                                                                                                                                                                                                      |

Ref. del material: [pinecone.io/learn/vector-embeddings](https://pinecone.io/learn/vector-embeddings)

## 4. RAG — arquitectura base

**RAG** (*Retrieval-Augmented Generation*, generación aumentada por recuperación) combina dos capacidades: **recuperar** información relevante desde una base de datos/documentos externos, y **generar** texto con un LLM (*Large Language Model*, modelo de lenguaje de gran escala) usando esa información recuperada como contexto adicional. Esto permite responder con datos actualizados y específicos, incluso si no estaban en el entrenamiento original del modelo.

El material cita directamente el paper que originó el término:

**Lewis, P. et al. — "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", NeurIPS 2020. [arXiv:2005.11401](https://arxiv.org/abs/2005.11401)**

El paper combina memoria paramétrica (un modelo *seq2seq* preentrenado) con memoria no-paramétrica (un índice vectorial denso de Wikipedia, accedido con un retriever neuronal entrenado) y demuestra que este enfoque genera respuestas más específicas, diversas y factuales que un modelo puramente paramétrico — estableciendo el estado del arte en tareas de *question answering* de dominio abierto.

**El flujo RAG básico, reconstruido del diagrama del material:**

User —Query→ Embedding → Vector DB → Retrieved Info + Query + System Prompt → LLM → Output

## 4.1 Chunking — cómo se dividen los documentos

El **chunking** es el proceso de dividir documentos largos en fragmentos (*chunks*) más pequeños para facilitar su indexación, búsqueda y recuperación. Una buena estrategia de chunking mejora directamente la precisión de los resultados.

| Estrategia                              | Qué hace                                                                                     | Trade-off                                                                                   |
|-----------------------------------------|----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Tamaño fijo                             | Divide en fragmentos de tamaño constante (ej. 500 tokens)                                    | Fácil y eficiente, pero puede cortar ideas a la mitad                                       |
| Semántico                               | Usa un modelo de lenguaje para identificar límites naturales (párrafos, secciones temáticas) | Preserva significado; ideal para documentos técnicos/legales/científicos                    |
| Con solapamiento ( <i>overlapping</i> ) | Los fragmentos se superponen parcialmente (ej. 500 tokens con 100 de solape)                 | Mejora la continuidad de contexto entre chunks; reduce pérdida de información en los bordes |
| Jerárquico                              | Divide en niveles: capítulos → secciones → párrafos                                          | Permite búsquedas estructuradas; útil para bases de conocimiento extensas                   |
| Dinámico basado en embeddings           | Agrupar frases/párrafos de alta similitud semántica usando embeddings vectoriales            | Chunks coherentes en significado aunque no sean contiguos en el texto original              |

### Investigación complementaria — dos estrategias que el diagrama del material menciona pero el texto no desarrolla:

- **Agentic chunking**: en vez de una regla fija, un agente (LLM) decide dinámicamente dónde cortar cada documento según su propio criterio sobre qué constituye una unidad de información completa.
- **Late chunking** (Jina AI, 2024): invierte el orden habitual — primero se genera el embedding del documento **completo** (aprovechando que los modelos contextuales "ven" todo el texto), y **después** se extraen los embeddings por chunk a partir de esa representación global ya contextualizada, en vez de embeber cada chunk de forma aislada. Esto evita perder contexto que normalmente se pierde al cortar el documento *antes* de generar embeddings.

## 4.2 Evaluación de RAG — RAGAS

El material presenta **RAGAS** (*RAG Assessment*, evaluación de RAG — [docs.ragas.io](https://docs.ragas.io)), un framework de métricas específico para sistemas RAG, agrupadas según qué datos requieren:

| Grupo                                                 | Métrica                                       | Qué mide                                                                                                                      | Datos requeridos                         |
|-------------------------------------------------------|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| Sin referencia                                        | Faithfulness (fidelidad)                      | Nº de afirmaciones en la respuesta generada que están respaldadas por el contexto recuperado, sobre el total                  | Respuesta, Contextos                     |
|                                                       | Answer Relevance (relevancia de la respuesta) | Qué tan relacionada está la respuesta generada con la pregunta                                                                | Respuesta, Contextos, Pregunta           |
|                                                       | Context Relevancy (relevancia del contexto)   | Proporción de oraciones del contexto recuperado que son relevantes para responder                                             | Contextos, Pregunta                      |
| Basadas en <i>Ground Truth</i> (verdad de referencia) | Context Precision                             | Si los chunks relevantes para el Ground Truth están rankeados alto                                                            | Contextos, Pregunta, Ground Truth        |
|                                                       | Context Recall                                | Nº de oraciones del Ground Truth que se pueden atribuir al contexto recuperado, sobre el total                                | Contextos, Ground Truth                  |
|                                                       | Context Entities Recall                       | Fracción de entidades recuperadas del Ground Truth                                                                            | Contextos, Ground Truth                  |
|                                                       | Answer Semantic Similarity                    | Similitud semántica (cross-encoder) entre el Ground Truth y la respuesta                                                      | Respuesta, Ground Truth                  |
|                                                       | Answer Correctness                            | Alineación entre la respuesta generada y el Ground Truth — promedio ponderado de similitud semántica y similitud factual (F1) | Respuesta, Ground Truth                  |
|                                                       | Harmfulness, Maliciousness, Coherence,        | Evaluaciones cualitativas tipo LLM-as-judge sobre daño potencial,                                                             | Respuesta (+ Contexto/Pregunta opcional) |
| Aspect Critique (sin referencia)                      |                                               |                                                                                                                               |                                          |

| Grupo | Métrica                     | Qué mide                                              | Datos requeridos |
|-------|-----------------------------|-------------------------------------------------------|------------------|
|       | Correctness,<br>Conciseness | coherencia, corrección y<br>concisión de la respuesta |                  |

**Por qué esta tabla importa en la práctica:** RAGAS deja explícito que evaluar RAG no es una sola métrica — hay que decidir primero si tienes o no un *Ground Truth* etiquetado (un dataset de referencia con las respuestas "correctas"), porque eso determina qué familia de métricas puedes calcular. Sin Ground Truth, solo tienes las métricas "sin referencia" (Faithfulness, Answer Relevance, Context Relevancy).

**Práctica del laboratorio referenciada:** el material muestra una captura de **LangSmith** (plataforma de observabilidad de LangChain) con un experimento de evaluación RAG real — una tabla comparando Input / Reference Output / Output con columnas de score Hallucination y Helpful por cada fila, más latencia y tokens — el patrón práctico de correr RAGAS (o evaluadores similares) sobre un dataset de pares pregunta-respuesta, vía el tutorial [docs.langchain.com/langsmith/evaluate-rag-tutorial](https://docs.langchain.com/langsmith/evaluate-rag-tutorial).

## 4.3 Laboratorio práctico — Chroma + LangChain

El material referencia el ejercicio "RAG with Chroma and LangChain" — **Chroma** es una base de datos vectorial de código abierto, comúnmente usada en tutoriales de RAG con LangChain por su simplicidad de instalación local (sin necesidad de infraestructura en la nube).

## 5. Agentic RAG — cuando el agente decide cómo y cuándo recuperar

El material presenta un diagrama comparativo de tres arquitecturas, en orden de creciente autonomía:

RAG clásico:

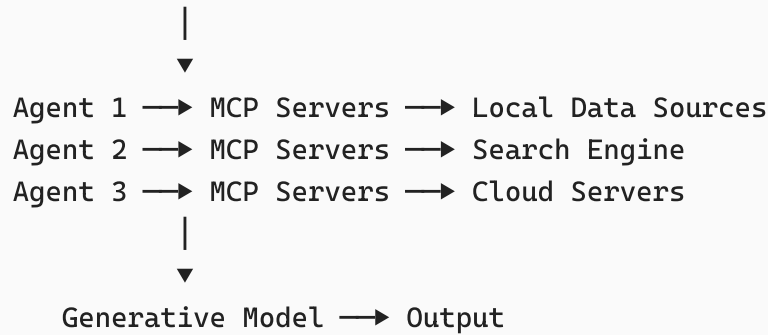
```
User —Query→ [Embedding → Vector DB] → Retrieved Info + Query + System Prompt → LLM → Output
(retrieval = UN solo paso fijo, siempre antes de generar)
```

AI Agent (sin RAG):

```
User —Query→ Agent [Memory + Planning] → Tools → Data Sources → Output
(el agente decide qué tool usar, pero no hay recuperación semántica dedicada)
```

Agentic RAG:

User —Query→ Aggregator Agent [Short/Long Term Memory + Planning (ReACT/CoT)]



**La diferencia conceptual clave:** en RAG clásico, la recuperación es **un paso fijo y único** — siempre se recupera una vez, antes de generar, sin importar si la consulta lo necesita o no. En **Agentic RAG**, un agente orquestador (equipado con memoria de corto/largo plazo y una estrategia de planificación explícita — **ReACT**, *Reasoning + Acting*, o **CoT**, *Chain of Thought*, razonamiento en cadena) puede:

- Decidir **si** recuperar información o no.
- Decidir **de qué fuente** recuperar, delegando a sub-agentes especializados (cada uno conectado a distintas fuentes vía **MCP** — *Model Context Protocol*, protocolo estándar para que un modelo/agente consuma herramientas y fuentes de datos externas de forma uniforme — a servidores locales, motores de búsqueda, o servicios en la nube).
- Iterar: recuperar, evaluar si la información alcanza, y volver a recuperar si hace falta — en vez de un único intento.

**Conexión con la Sesión 9:** este diagrama es, en esencia, la arquitectura de **5 memorias** (§14 del análisis de la Sesión 9) aplicada específicamente al problema de recuperación — el "Aggregator Agent" combina Memoria de Corto/Largo Plazo con una estrategia de planificación (Procedimental) para coordinar múltiples fuentes de Memoria Semántica en paralelo.

Ref. del material: diagrama de @rakeshgohe101.

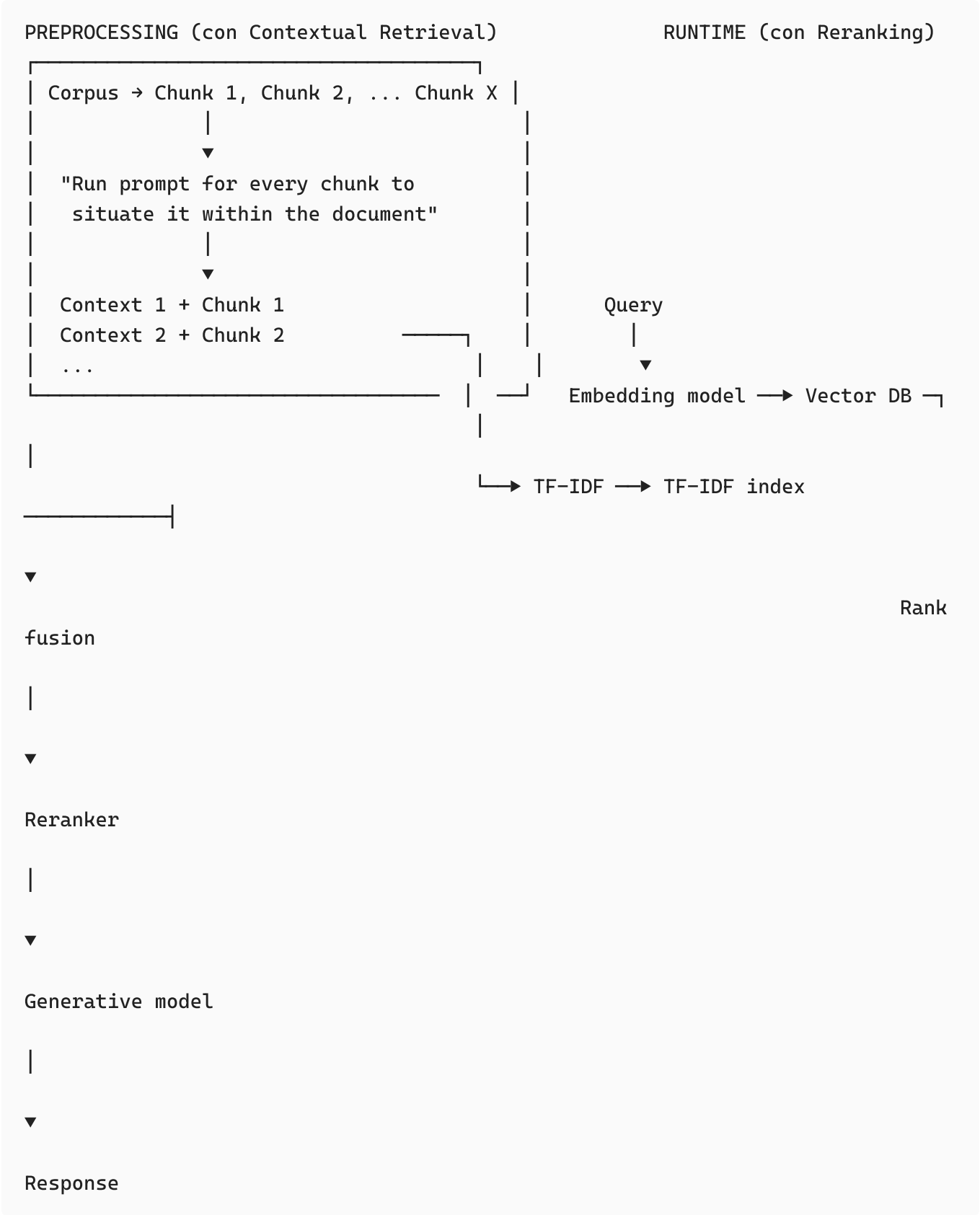
## 6. Contextual Retrieval — resolver la pérdida de contexto del chunking

El material define: "*Contextual Retrieval is a preprocessing technique that improves retrieval accuracy*" — una técnica de **preprocesamiento** (no de arquitectura de consulta) que ataca directamente el problema central del chunking (§4.1): cuando se divide un documento en fragmentos, cada chunk **pierde el contexto** del documento completo que le daba sentido (ej.



un chunk que dice *"la tasa subió un 3% ese trimestre"* sin decir de qué empresa ni qué trimestre).

**El diagrama "Combined" del material, reconstruido:**



Los dos mecanismos que se combinan:

- 1. **Contextualización de cada chunk**: antes de indexar, se le pide a un LLM que genere una breve descripción de cómo ese chunk específico se ubica dentro del documento completo, y esa descripción se antepone al chunk ( Context + Chunk ) antes de embeberlo. Así, el embedding del chunk ya "sabe" de qué documento y sección viene.
- 2. **Búsqueda híbrida con rank fusion**: se indexa el mismo contenido contextualizado tanto en un embedding vectorial (similitud semántica) como en un índice **TF-IDF** (coincidencia léxica exacta de términos) — y en tiempo de consulta se combinan (*fusionan*) ambos rankings antes de aplicar un **reranker** final, que reordena el *top-N* con un modelo más preciso (y más costoso) antes de pasarlo al modelo generativo.

**Investigación complementaria — el origen real de esta técnica**: el diagrama corresponde a **"Contextual Retrieval"**, técnica publicada por **Anthropic** en septiembre de 2024 ([anthropic.com/news/contextual-retrieval](https://anthropic.com/news/contextual-retrieval)). Anthropic reportó que combinar *contextual embeddings* + *contextual BM25* reduce la tasa de fallos de recuperación en ~49% frente a RAG estándar, y hasta ~67% al agregar el paso de reranking — la mejora más grande viene de dar contexto a los chunks *antes* de embeberlos, no del reranking por sí solo.

6.1 Hybrid Search — la pieza de "Rank Fusion"

El material dedica una diapositiva aparte a explicar el componente de búsqueda híbrida que alimenta el "Rank Fusion" de arriba:

*"A Standard Retrieval-Augmented Generation (RAG) system that uses both embeddings and Best Match 25 (BM25) to retrieve information. TF-IDF (term frequency-inverse document frequency) measures word importance and forms the basis for BM25."*

**BM25** (*Best Match 25*) es una función de ranking basada en TF-IDF, ampliamente usada en motores de búsqueda léxica (ej. Elasticsearch) — a diferencia de un embedding, BM25 encuentra coincidencias **exactas de términos**, lo cual es complementario a la búsqueda semántica: un embedding puede fallar en encontrar un código de producto o un nombre propio exacto que BM25 sí encuentra directamente.

6.2 Técnicas adicionales de optimización de recuperación

El material (citando a *Chip Huyen, AI Engineering*) agrega tres técnicas más, más allá de Contextual Retrieval:

| Técnica         | Qué hace                                                                      | Advertencia del material                     |
|-----------------|-------------------------------------------------------------------------------|----------------------------------------------|
| Query Rewriting | Enriquece la consulta inicial con información contextual antes de buscar (ej. | Puede introducir latencia — evaluar antes de |

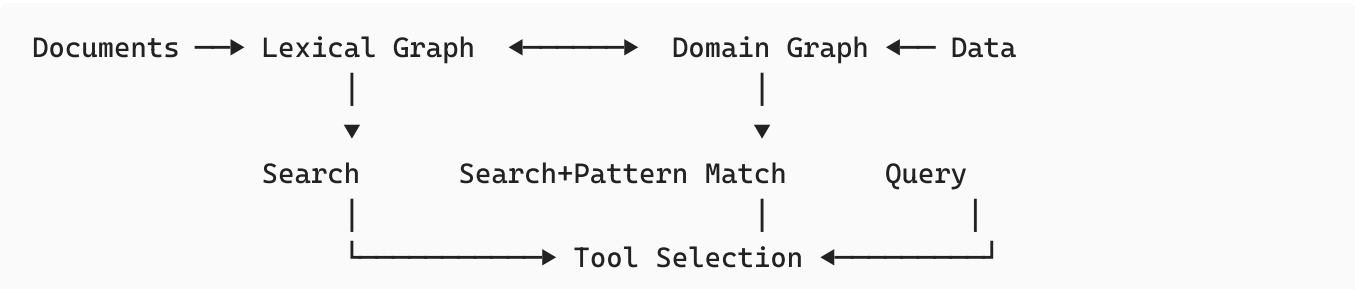
| Técnica                                              | Qué hace                                                                                                                                                                          | Advertencia del material                                                                 |
|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
|                                                      | "¿y ella?" → "¿y la tía Mabel de la pregunta anterior?")                                                                                                                          | implementar                                                                              |
| <b>Contextual Retrieval</b><br>("chunks-for-chunks") | Cada chunk recuperado dispara recuperaciones adicionales de contexto suplementario (tags relacionados, metadata asociada)                                                         | —                                                                                        |
| <b>Hybrid Search</b>                                 | Combina recuperación por término (ej. Elasticsearch) con recuperación por embedding, típicamente obteniendo primero ~50 documentos por término y luego re-rankeando por embedding | Patrón típico: recuperación amplia y barata primero, refinamiento preciso y caro después |

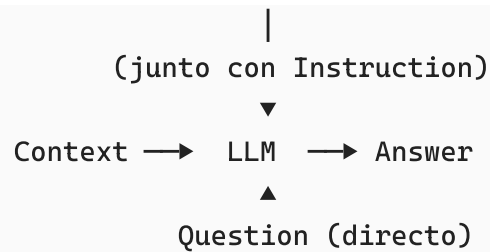
## 7. GraphRAG — cuando la estructura de relaciones importa más que la similitud

El material introduce **GraphRAG** solo con el nombre y una visualización (un grafo de nodos Document / Chunk interconectados, generado con **Neo4j**, una base de datos de grafos). La idea central, que el diagrama no explica en texto pero se investiga aquí:

**Investigación complementaria — GraphRAG (Microsoft Research, 2024):** el RAG vectorial estándar (§4) responde bien preguntas puntuales ("¿cuál es la política de cancelación?") pero falla en preguntas **globales o agregadas** sobre todo el corpus ("¿cuáles son los temas principales across todos los documentos?"), porque la similitud vectorial busca fragmentos parecidos a la pregunta, no una síntesis del conjunto. GraphRAG resuelve esto extrayendo, con un LLM, **entidades y relaciones** de los documentos para construir un grafo de conocimiento, agrupando el grafo en comunidades temáticas y generando resúmenes jerárquicos de cada comunidad — permitiendo responder tanto preguntas puntuales (navegando el grafo) como preguntas globales (usando los resúmenes de comunidad).

Una segunda diapositiva del material (sin la etiqueta "GraphRAG" explícita, pero temáticamente relacionada) muestra una arquitectura de grafo con dos capas:





**Lectura de este segundo diagrama:** distingue un **Lexical Graph** (relaciones extraídas directamente del texto — menciones, co-ocurrencias) de un **Domain Graph** (el modelo de dominio del negocio — entidades y relaciones curadas), ambos dentro de la misma base de datos de grafos. Según el tipo de pregunta, el sistema elige entre tres modos de acceso ( `Search` léxico simple, `Search + Pattern Match` combinando texto con patrones estructurales del grafo, o `Query` estructurada tipo Cypher) antes de ensamblar el contexto final para el LLM — un patrón de **selección dinámica de estrategia de recuperación**, análogo en espíritu al `memory_strategy` configurable de LangGraph visto en la Sesión 9.

## 8. Laboratorios y tarea de la sesión

| Actividad                       | Instrucción                                                                                                                                                         |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Lab — "Aterrizando a proyectos" | En grupo, elaborar un <i>draft</i> del proceso de RAG para los documentos propios del equipo: identificar los pasos necesarios y plasmarlos en un script de Python. |
| Tarea — RAG                     | Implementar un RAG en Python con <b>Chroma</b> que resuelva un caso personal. Entregable: Google Doc con {descripción, código} .<br><b>Fecha límite: 02/08.</b>     |

## 9. Síntesis — lo que hay que llevarse de esta sesión

1. **Los embeddings tienen una jerarquía de sofisticación** (frecuencia → predicción → contexto), y los sistemas RAG modernos dependen específicamente de embeddings **contextuales** (BERT/GPT y variantes) para funcionar bien.
2. **Una base de datos vectorial es, en esencia, un motor de búsqueda por vecino más cercano aproximado** — el trade-off central es precisión vs. velocidad, resuelto con algoritmos de indexado (PQ, LSH, HNSW).
3. **RAG combina recuperación + generación** para responder con información actualizada y específica sin reentrenar el modelo — el paper fundacional (Lewis et al., 2020) sigue siendo la referencia académica del patrón.

4. **El chunking es una decisión de diseño con trade-offs reales**, no un detalle de implementación — la estrategia elegida (fija, semántica, con solapamiento, jerárquica, dinámica) determina directamente la calidad de la recuperación.
  5. **Evaluar RAG requiere elegir primero si hay o no un Ground Truth** — RAGAS separa métricas sin referencia (Faithfulness, Answer Relevance, Context Relevancy) de métricas que sí lo requieren (Context Precision/Recall, Answer Correctness).
  6. **Agentic RAG reemplaza el "recuperar una vez, siempre" por una decisión activa del agente**: si recuperar, de qué fuente (vía sub-agentes/MCP), y si iterar cuando el primer intento no alcanza.
  7. **Contextual Retrieval ataca la pérdida de contexto del chunking en el preprocesamiento**, no en la consulta: situar cada chunk dentro de su documento antes de embeberlo, combinado con búsqueda híbrida (embeddings + BM25) y reranking, es la técnica con mayor impacto medido (Anthropic, 2024) para reducir fallos de recuperación.
  8. **GraphRAG resuelve lo que el RAG vectorial no puede**: preguntas globales/agregadas sobre todo un corpus, a costa de la complejidad adicional de construir y mantener un grafo de conocimiento.
- 

## 10. Checklist práctico — diseñando tu propio sistema RAG

- ☐ ¿Qué familia de embeddings vas a usar? (Para RAG moderno, casi siempre contextual — BERT/GPT y derivados, no frecuencia ni predicción pura.)
- ☐ ¿Qué estrategia de chunking corresponde a tu tipo de documento? (Fija para contenido homogéneo corto; semántica o jerárquica para documentos largos/estructurados.)
- ☐ ¿Vas a contextualizar cada chunk antes de embeberlo (Contextual Retrieval), o vas a indexar los chunks "en crudo"? (La ganancia de precisión reportada es significativa — evalúa el costo de preprocesamiento contra el beneficio.)
- ☐ ¿Necesitas búsqueda híbrida (embeddings + BM25/TF-IDF), o la similitud semántica sola es suficiente para tu caso? (Términos exactos — códigos, nombres propios — se benefician de BM25.)
- ☐ ¿Vas a agregar un paso de reranking antes de pasar el contexto al LLM?
- ☐ ¿Tu caso de uso necesita RAG simple (una recuperación fija por consulta), o Agentic RAG (el agente decide si/cuándo/de dónde recuperar, con múltiples fuentes)?
- ☐ ¿Tus preguntas son mayormente puntuales (RAG vectorial estándar es suficiente) o también necesitas responder preguntas globales/agregadas sobre todo el corpus (considera GraphRAG)?
- ☐ ¿Cómo vas a evaluar tu sistema? ¿Tienes un Ground Truth etiquetado, o vas a depender de métricas sin referencia (RAGAS)?

---

## 11. Referencias

### Del material original:

- Diagramas propios del curso — embeddings, Vector Database, chunking, RAG vs. Agentic RAG (@rakeshgohel01), Contextual Retrieval "Combined", GraphRAG (visualización Neo4j).
- Lewis, P. et al. — *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020. [arXiv:2005.11401](https://arxiv.org/abs/2005.11401)
- RAGAS — framework de evaluación de RAG. [docs.ragas.io](https://docs.ragas.io)
- LangSmith — tutorial de evaluación de RAG. [docs.langchain.com/langsmith/evaluate-rag-tutorial](https://docs.langchain.com/langsmith/evaluate-rag-tutorial)
- Pinecone — recurso de aprendizaje sobre vector embeddings y bases de datos vectoriales. [pinecone.io/learn/vector-embeddings](https://pinecone.io/learn/vector-embeddings)
- Chip Huyen — *AI Engineering* (referencia citada para técnicas de optimización de recuperación: Query Rewriting, Contextual Retrieval, Hybrid Search).
- LangChain — integración de *document loaders* y RAG con Chroma. [python.langchain.com/docs/integrations/document\\_loaders](https://python.langchain.com/docs/integrations/document_loaders)

### Investigación complementaria (añadida en este documento):

- Anthropic — *Introducing Contextual Retrieval*, septiembre 2024. Técnica de contextualización de chunks + BM25 contextual + reranking; origen real del diagrama "Combined" del material.
  - Microsoft Research — *GraphRAG*, 2024. Extracción de entidades/relaciones vía LLM para construir un grafo de conocimiento y resumir por comunidades, resolviendo preguntas globales que el RAG vectorial no puede responder bien.
  - Jina AI — *Late Chunking*, 2024. Estrategia de chunking que embebe el documento completo antes de extraer los embeddings por fragmento, preservando contexto global.
  - Arco interno del curso: Sesión 9 (Módulo 4) — Memoria Contextual de Agentes, cuya arquitectura de 5 memorias (Episódica/Semántica/Procedimental/Corto/Largo Plazo) es el marco conceptual que explica la memoria del "Aggregator Agent" en el diagrama de Agentic RAG (§5).
-

*Documento generado a partir del PDF de la Sesión 13 (Módulo 5, UTEC Posgrado) — texto extraído + diapositivas gráficas interpretadas visualmente — más investigación propia sobre RAG, Contextual Retrieval, GraphRAG y Late Chunking.*